General Submission Instructions

- Read the instructions carefully before starting the assignment.
- The practical part is submitted once per group, as described in Problem 1. Only one group member should submit the practical part.
- Each theoretical question must be submitted individually (to clarify: answers should be your own, do not work on them with other students) as a single PDF file only. Do not submit DOCX, JPG, PNG, or other formats.
- The theoretical submission file should be named:

  `ex1t_<ID>_Q<question-number>.pdf`

  where `<ID>` is your ID number and `<question-number>` is the number of the theoretical question submitted.

  For example:

  `ex1t_123456789_Q2.pdf`

  `ex1t_123456789_Q3.pdf`

- If handwritten answers are submitted, they must be very clear and legible.
- All required outputs, printed tables, previews, and analyses must be included in the consolidated report, not only saved as files.
- Any plots included in the submission must be informative and easy to read. Each plot should include a title, axis labels, and a legend when needed.
- Make sure you follow the News forum and HW forum for any updates or clarifications.
- Points may be reduced for submissions that do not follow the required file names, folder structure, or formatting instructions.

Practical Part: Creepy Crawlers (Group Submission)

Problem 1 (Creepy Crawlers).

In this question, you will use crawling to create a (tiny) dataset.
Crawling websites is impolite in general, so MAKE SURE you are following these rules:

- Start by downloading a single page to your computer. Only when you have it right (that is, you can open the local file, parse the html and save the output in the format you want), you can add a loop and crawl multiple pages.
- Make sure there is a **significant delay** between each request to the website (several seconds should do the trick).
- Be **really** careful. :)

General instructions: The site you need to crawl is `https://www.bookdelivery.com/`. For the sake of this homework, we only need you to crawl the **first 5 pages from each top-level category (Art, Law, etc.)** (if there are fewer than 5 pages in a category, crawl them all), but your code should be able to crawl the entire site, if we remove that restriction. (In other words, do not hand-code URLs you want to crawl).

You can use existing code for a web crawler/scraper (you can use LLMs, they are very helpful for crawling). A part of the assignment is experimenting with tools, so we will not recommend specific ones (but we do recommend not trying to write regular expressions for extracting content from html, if html is too convoluted – use Beautiful Soup, lxml, etc.).

Links you might find useful:

- `https://www.w3.org/MarkUp/Guide/` A gentle intro to HTML (totally outdated, but if you haven't touched HTML before it's a decent start)
- `https://www.digitalocean.com/community/tutorials/how-to-scrape-web-pages-with-beautiful-soup-an`
- `https://www.digitalocean.com/community/tutorials/how-to-crawl-a-web-page-with-scrapy-and-python`
- `https://www.selenium.dev/` (web browser automation, especially useful for pages with dynamic content)
- `https://towardsdatascience.com/how-to-build-a-simple-web-crawler-66082fc82470`

Have fun. :)

Step 1: Crawling the data

Write a Python script to crawl and collect book data:

1. Start your crawl at `https://www.bookdelivery.com/`
2. Programmatically extract all category links from the homepage (do not manually copy links; your solution must work even if the site structure changes).
3. For each category:
   - Traverse the first 5 pagination pages (if there are fewer, traverse them all).
   - Extract links to individual book pages.
4. For each book page, your code should:
   - Visit the page using `requests`.
   - Parse the HTML (e.g., with `BeautifulSoup`).

- Extract the following data:
  - Title
  - Category (where you crawled it from)
  - Categories (comma-separated)
  - Authors (comma-separated)
  - Price in NIS (2 decimal digits, rounded up)
  - Price in USD (use an exchange rate of 3.01, 2 decimal digits, rounded up)
  - Year
  - Synopsis
  - Synopsis length (number of characters)
  - StarRating ("None" if there are 0 reviews. Compute from Star Distribution, 2 decimal digits, rounded up)
  - NumberOfReviews (it is ok if it is 0)
  - Language
  - Format
  - Dimensions (numerical, comma-separated)
  - Dimensions unit (e.g., inch)
  - Weight (number)
  - Weight unit
  - ISBN/ISBN13

Step 2: DataFrame Construction

Load all extracted data into a Pandas DataFrame named `df_books`.

1. Cast all numeric fields to appropriate numeric types.
2. Save the raw DataFrame to:
   - `output/books_raw.csv`
   - `output/books_raw.json`
3. In addition, attach **one example**: a **screenshot** of one of the pages you crawled, plus the corresponding JSON:
   - `output/books_example.json`
   - `output/books_example.jpg`

JSON file format:

```
{
  "records": {
    "record": [
      {
        "id": "1",
        "url": "http://something",
        "field1": "value 1",
        "field2": "value 2"
      },
      {
        "id": "2",
        "url": "http://something",
        "field1": "value 1"
```

```
      },
      {
        "id": "3",
        "url": "http://something",
        "field1": "value 1",
        "field2": "value 2"
      }
    ]
  }
}
```

Note that if a book page did not include a certain field, the JSON does not include it as well.

Step 3: Preview

1. Print the first 10 rows of `df_books` before sorting by `Title`.
2. Sort the DataFrame by `Title` in ascending order and print the first 10 rows again.
3. Save both outputs to:
   - `output/books_before_sort.csv`
   - `output/books_after_sort.csv`

Step 4: (slightly) More Data Processing

Add a binary feature `IsExpensive = 1` if `Price in NIS` is greater than the median `Price in NIS` in the data you crawled, and `0` otherwise.

Also add a `NumberOfAuthors` feature.

Save the processed DataFrame to:

- `output/books_processed.csv`

Update the JSON file by adding the derived features and save it as:

- `output/books_processed.json`

Print the first 10 rows after processing and save them to:

- `output/books_processed_preview.csv`

Step 5: Summary Statistics

For the following columns:

- `Price in USD`
- `Year`
- `StarRating`
- `NumberOfReviews`
- `NumberOfAuthors`

Compute, only for books where a value appears in the column: mean, standard deviation, min, max, and median.

Also include the total number of rows in the data.

Save to:

- `output/books_summary.csv`

Practical Part Submission Requirements

Only one group member should submit the practical part.

Submit a single ZIP file named:

`ex1p_<ID>.zip`

where `<ID>` is the ID number of the student submitting the practical part.

The ZIP file should contain:

`code/` folder:

- `books_crawler.py`
- Any additional Python files necessary for running the assignment
- `README.txt`, if needed, with execution instructions

`output/` folder:

- `books_raw.csv`, `books_raw.json`, `books_example.json`, `books_example.jpg`, `books_before_sort.csv`, `books_after_sort.csv`, `books_processed.csv`, `books_processed.json`, `books_processed_preview.csv`, `books_summary.csv`

`consolidated_report.pdf` should include:

- The names and ID numbers of all group members
- The first 10 rows before sorting, shown as a table in the PDF
- The first 10 rows after sorting, shown as a table in the PDF
- The first 10 rows after processing, shown as a table in the PDF
- The summary statistics table
- Clear references to the corresponding files saved in the `output/` folder

Submission Checklist

Before submitting, verify that:

- Crawling is fully programmatic.
- No URLs were manually collected.
- All requested pages were traversed (categories, pagination, books).
- Numeric fields are correctly converted.
- Derived features are correct.
- JSON format follows the required structure specified in the assignment, with records stored under the `records` → `record` fields.
- The `code/` folder contains the required Python scripts and any additional files needed to run the assignment.
- The `output/` folder contains all required output files.
- The consolidated report includes all required tables and outputs as tables in the PDF, not only as console logs.

Theoretical Part: Individual Submission (your own work)

Problem 2 (Clustering).

1. Let $X = \{x_1, \ldots, x_N\} \subseteq \mathbb{R}^d$ be a dataset, and suppose that all points satisfy

$$\|x_i\| = 1.$$

Thus, all the points lie on a unit sphere. Consider the following two clustering algorithms with k clusters:

Algorithm A: Ordinary K-Means	**Algorithm B: Normalized-Center K-Means**
Input: X, k	**Input:** X, k
Initialize centers $\mu_1, \ldots, \mu_k \in \mathbb{R}^d$	Initialize centers $\mu_1, \ldots, \mu_k \in \mathbb{R}^d$ with $\|\mu_j\| = 1$
Repeat until convergence:	**Repeat until convergence:**
1. Assign each x_i to cluster: $\arg\min_j \|x_i - \mu_j\|^2$	1. Assign each x_i to cluster: $\arg\max_j x_i^\top \mu_j$
2. Update each center μ_j to the arithmetic mean of its points.	2. Update each center μ_j to the arithmetic mean of its points.
	3. Normalize each updated center: $\mu_j \leftarrow \mu_j / \|\mu_j\|$

   (a) Show that for unit vectors x_i, minimizing $\|x_i - \mu_j\|^2$ over unit-norm centers is equivalent to maximizing $x_i^\top \mu_j$.

   (b) Will Algorithm A and Algorithm B always return the same clustering? Will your answer change if all data points are normalized? Explain.

2. Let $X = \{x_1, \ldots, x_N\} \subseteq \mathbb{R}^d$ be a finite dataset of size N. Fix two integers $t, s \geq 1$.

For each point $x_i \in X$, let $N_t(x_i)$ denote the set of its t nearest neighbors in $X \setminus \{x_i\}$ by Euclidean distance.

Define a similarity graph $G = (V, E)$ by setting $V = X$, and for any two distinct points $x_i, x_j \in X$, define

$$(x_i, x_j) \in E \quad \Longleftrightarrow \quad |N_t(x_i) \cap N_t(x_j)| \geq s.$$

The clustering returned by the algorithm is the partition of X into the connected components of G.

   (a) Suppose $s = 1$, $t = N - 1$, $N \geq 3$. Characterize the number of clusters.

   (b) Suppose that $s > t$. Characterize the number of clusters.

   (c) Let $G_{t,s}$ denote the graph constructed by the algorithm with parameters (t, s). Prove or disprove each of the following statements:

      i. If $t_1 \leq t_2$, then $E(G_{t_1,s}) \subseteq E(G_{t_2,s})$.

      ii. If $s_1 \leq s_2$, then $E(G_{t,s_2}) \subseteq E(G_{t,s_1})$.

   (d) Offer a dataset for which this algorithm can find a meaningful partition and K-Means can not. Explain your answer.

3. Let $X = \{x_1, \ldots, x_N\} \subseteq \mathbb{R}^d$ be a dataset, and fix parameters $\varepsilon > 0$ and $\text{MinPts} \in \mathbb{N}$.

For a point $x \in X$, define its ε-neighborhood by

$$N_\varepsilon(x) = \{y \in X : \|y - x\| \leq \varepsilon\}.$$

Recall the following definitions:

- A point x is called a *core point* if $|N_\varepsilon(x)| \geq \text{MinPts}$.
- A point y is *directly density-reachable* from a core point x if $y \in N_\varepsilon(x)$.
- Two points are in the same cluster if they are connected by a chain of direct density-reachability relations through core points, with border points attached to nearby core points.

Consider the clustering algorithm DBSCAN with parameters $(\varepsilon, \text{MinPts})$.

(a) Show that if $\text{MinPts} = 1$, then every point is a core point. Conclude that DBSCAN reduces to finding the connected components of the graph with edge set

$$E = \{(x_i, x_j) : \|x_i - x_j\| \leq \varepsilon\}.$$

(b) What is the result of DBSCAN when $\text{MinPts} > N$?

(c) A student claims: "If I have two DBSCAN clusterings of the same data, and I know that ε of the first is larger than ε of the second, then the number of clusters in the first clustering cannot be higher than in the second clustering." Is this always true? Prove it or give a counterexample.

(d) A student claims: "If we increase MinPts while keeping ε fixed, then every cluster found with the new parameter must be contained in some old cluster." Is this always true? Prove it or give a counterexample.

Problem 3 (Hujinator).

Part 1 : The Quest of the Hujinator

Context

Deep within the server rooms of Givat Ram, there exists a legendary entity known as the **Hujinator**. Its mission is to decode the complex social fabric of HUJI.
Currently, the Hujinator is facing a crisis of logic. Its most popular feature the **'Beloved Professor Predictor'** has been corrupted. This tool was designed to analyze a teacher's profile (their department, teaching scores, even their coffee intake) and predict if they would win the hearts of the students.

Your task is to help the Hujinator choose the best questions to ask by calculating Entropy and Information Gain.

The Dataset

Below is a sample of 10 individuals from the university. We want to predict if an individual is **"Beloved by Students"** .

Nom	TeachCS (A)	GiveBonus (B)	HasGlasses (C)	Beloved
Prof 1	1	1	1	**Y**
Prof 2	1	1	1	**Y**
Prof 3	1	0	1	**N**
Prof 4	1	0	0	**N**
Prof 5	0	1	1	**Y**
Prof 6	0	1	0	**Y**
Prof 7	0	0	0	**N**
Prof 8	0	0	0	**N**
Prof 9	1	1	1	**Y**
Prof 10	0	1	1	**Y**

Questions

1. **Initial Entropy $H(S)$:** Calculate the initial entropy of the entire dataset (S). Show your formula and steps.

2. **Information Gain (IG):** Calculate the Information Gain for each of the three attributes:

   - Gain(S, A) : Teaches CS
   - Gain(S, B) : Gives Bonuses
   - Gain(S, C) : Wears Glasses

   And explain the result you have found.

3. **Tree Construction:** Based on your calculations in Question 2, which attribute will the Hujinator choose as the Root Node? Draw the resulting Decision Tree

4. Suppose the University Administration bans all "Bonuses" (Attribute B is removed). Between Attributes A and C, which one would the Hujinator pick now? Justify your answer using your previous calculations.

Part 2: The Junior Intern's Incident

Background: Incident Report #2026-04

Disaster has struck the HUJI Computer Science department! A junior intern, fueled by too much coffee and not enough sleep, used an LLM, did not check the output, and performed a "Force Push" to the production branch. This catastrophic update corrupted the core logic of the **HUJINATOR**.

It seems like the LLM code is broken, and the production script hujinator.py is now generating nonsensical trees. The original senior developer is currently hiking the Israel National Trail and is unreachable.

Your mission is to perform a **Post-Mortem Analysis** and **fix the LLM code**. You are provided with the corrupted script and the dataset. You must identify all logical, mathematical, and structural errors introduced by the LLM and restore the HUJINATOR to its former glory.

Provided Files

- `hujinator.py`: The buggy script that needs fixing.
- `visualisation.py`: Helper functions to generate the tree's visual map.
- `hujinator_1000.csv`: A database containing 1,000 entries of professor data.

Task A: Bug Report & Diagnostics

You must identify and fix the errors in the provided `hujinator.py`. For each bug found, you must include in your report:

1. **Location:** The specific line number(s) (e.g., *Line 10 to 12 - Mathematically incorrect formula*).

2. **Diagnostic:** Explain why the current logic is mathematically or computationally flawed.

3. **Correction:** Fix the bug in the code.

Task B: Restoration & Validation

After fixing the code, run your script using the `hujinator_1000.csv` dataset.

1. **Optimization**: Ensure your code compiles, runs, and that your final tree is **minimal**: If a split leads to branches that all result in the same verdict, that split should not exist.
2. **Visual Evidence**: Include the corrected `hujinator_tree.png` generated by your fixed code.
3. **Predictor Logic**: Provide a brief explanation of the "Hujinator" predictor you came up with. What does the final tree tell us about the features that make a professor "Beloved"?

Submission Instructions - Problem 3

1. `hujinator.py`: Your corrected version of the script.
2. **A PDF containing:**
   - Your written answers for **Part 1** (Theoretical questions).
   - The detailed list of the identified bugs from **Task A**.
   - The visualization of your final, corrected tree.
   - The explanation of the logic behind the Hujinator.

Important Rules

- **Zero-Crash Policy**: The script must run without errors. A script that crashes or fails to produce a tree will result in a severe penalty.
- **Signature Integrity**: Do not change the function signatures! You must keep the original function names.
- **Visualisation**: You are not strictly required to use the provided `visualisation.py` file, but your results must conform to the definition of a **minimal** tree.

Problem 4 (Meta). How long (in hours) did this assignment take? Please answer in the designated Moodle forum that will be opened for this question, not in your PDF.